Very high bloat is not normally observed in local development databases. Dead row versions might accumulate in a high volume production system with a very high amount of Updates and Deletes and with Autovacuum not tuned and using default conservative values. This problem has lessened in newer versions of PostgreSQL and may continue to lessen in versions in the future.

Bloat management is nonetheless a common challenge, so you'll work with it a bit more in a hands on fashion.

To better understand bloat in your local development PostgreSQL server, you'll disable Autovacuum and create a very high amount of updates and deletes. By measuring the estimated bloat percentage before and after these updates, you can start to see how bloat accumulates.

To get started, collect a bloat estimate of the users table. Use the pgsql-bloat-estimation/table/table_bloat.sql[11] script from GitHub by running the statement from psql. Add the line WHERE schemaname = 'public' AND tblname = 'users' just before the ORDER BY, to limit the rows to the public.users table. The bloat_pct shows the current estimated bloat percentage. Take note of the current value.

Next, simulate bloat by creating a lot of Updates to the table. Run the following statement from psql to create 10 million updates to the users table with unique values for each update.

sql/bloat_simulate_user_updates.sql

```
UPDATE users
SET first_name =
  CASE (seq % 2)
    WHEN 0 THEN 'Bill' || FLOOR(RANDOM() * 10)
    ELSE 'Jane' || FLOOR(RANDOM() * 10)
  END
FROM GENERATE_SERIES(1,1000000) seq
WHERE id = seq;
```

This statement generates one update per row, matching the Primary Key id column to a sequence value. If you've got around 20 thousand rows, you'll see around 20 thousand updates. Each row gets a unique string that's either Bill or Jane with a number added on. Run the statement a few times to generate more updates.

Periodically check the bloat_pct column by running the query from earlier. Run some more updates. You'll see the bloat_pct growing. With these heavy amounts of updates, you're causing bloat in the table and in the Indexes.

11. https://github.com/ioguix/pgsql-bloat-estimation